

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

SEMESTER FINAL EXAMINATION
DURATION: 3 HOURS

WINTER SEMESTER, 2024-2025
FULL MARKS: 150

CSE 4501: Operating Systems

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer all 6 (six) questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. Consider the seat reservation program shown in Code Snippet 1 where two threads run concurrently, each attempting to book an available seat.

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 int available_seats = 100;
5 void* book_seat(void* arg) {
6     if (available_seats > 0) {
7         usleep(10);
8         available_seats--;
9         printf("Seat booked! %d seats left.\n", available_seats);
10    } else {
11        printf("No seats available.\n");
12    }
13    return NULL;
14 }
15 int main() {
16     pthread_t t1, t2;
17     pthread_create(&t1, NULL, book_seat, NULL);
18     pthread_create(&t2, NULL, book_seat, NULL);
19     pthread_join(t1, NULL);
20     pthread_join(t2, NULL);
21     return 0;
22 }

```

Code Snippet 1: Pseudocode for Question 1

- a) Explain what a race condition is and identify the specific critical section in the code above. 4 + 4
(CO3)
(PO2)
 Provide a step-by-step execution order of the two threads that results in overbooking (e.g., starting with `available_seats = 1`, `available_seats` becoming -1), mentioning the values of the `available_seats` in each step.
- b) Rewrite the `book_seat` function using a POSIX mutex to eliminate the race condition. 10
(CO3)
(PO2)
2. a) A system uses a multilevel queuing system:
- Queue 1 (system processes): Priority 1 (highest), scheduled with Round Robin (RR) with quantum = 4 ms.
 - Queue 2 (user processes): Priority 2, scheduled with First Come First Serve (FCFS).
- Processes never move between queues. The following processes arrive at time 0:

Table 1: A table of processes for Question 2 with queue and burst time

Process	Queue	Burst time
P1	1	5
P2	2	10
P3	2	8
P4	1	3

- i. Explain in step by step how the scheduler chooses which process to run. Draw the Gantt chart and compute the average waiting time for these processes, showing the calculation of waiting time for each process. 5 + 8
(CO2)
(PO1)
- ii. What is the advantage of having multiple queues? 5
(CO1)
(PO1)
- b) Consider a mix of CPU-bound and I/O-bound processes where, CPU-bound process having long bursts, e.g., 50 ms and I/O-bound process having Short bursts, e.g., 2 ms, but frequent I/O waits. Consider the following three processes in Table 2 that arrive at time 0.

Table 2: A table of processes for Question 2.b with type and burst patterns

Process	Type	CPU Burst Pattern (ms)
P1	CPU-bound	Single CPU burst of 50 ms
P2	I/O-bound	Burst of 2 ms → I/O wait → burst of 2 ms → I/O wait → burst of 2 ms (total CPU 6 ms) 2
P3	I/O-bound	Burst of 2 ms → I/O wait → burst of 2 ms → I/O wait → burst of 2 ms (total CPU 6 ms) 2

Assume the I/O waits are long enough that the process becomes blocked and releases the CPU; after each I/O wait, the process rejoins the ready queue.

- i. Briefly explain how each of the following algorithms would handle such a mix. For each algorithm, mention the execution order with time and discuss the effect on the I/O-bound processes. 3 × 3
(CO2)
(PO3)
 - ii. What is convoy effect? Which algorithm from above is susceptible to it? 5
(CO1)
(PO1)
- 1) FCFS 2) RR
 3) SJF (non-preemptive)
3. a) Briefly explain blocking, Non-blocking I/O and Asynchronous I/O. 8
(CO1)
(PO1)
 - b) Discuss the journey of a I/O request through the operating system, starting from the user application making a system call to the point where the data is available in user space, by drawing the I/O request life-cycle. Include the roles of the followings in your diagram: device driver, kernel's device-status table, Direct Memory Access (DMA) controller (if used), interrupt handler. 12
(CO1)
(PO1)
 4. a) Define a file from an operating system perspective. Briefly explain the three common file structures. 7
(CO1)
(PO1)
6
 - b) For a file with UNIX permissions $rw\bar{x}r-x\bar{x}$, explain what each group of permissions means and who can perform which operations. (CO1)
(PO1)
6 + 4
 - c) Compare single-level, two-level, and tree-structured directory systems. What is the difference between an absolute path and a relative path? Give an example of each using the Linux file system. (CO1)
(PO1)

- d) Describe how linked allocation works. How does File Allocation Table (FAT) improve upon basic linked allocation. 3 + 5
(CO2)
(PO3)
5. A small computer system uses swapping with contiguous allocation. At time t_0 , the free memory holes are as shown (addresses in KB):

Table 3: A table for Question 5 showing free memory

Hole	Start	Size
H1	0	10
H2	18	12
H3	35	5
H4	48	8
H5	60	15

All other memory is occupied by processes. The system uses a linked list of memory segments sorted by address.

- a) A new process, P of size 9 KB arrives. Show how FirstFit, BestFit, and WorstFit would place P. For each algorithm, state the resulting hole list after allocation. 3 × 3
(CO2)
(PO3)
10
- b) After several allocations and terminations, the system suffers from external fragmentation. Explain what external fragmentation is and why it occurs with contiguous allocation. The system administrator considers using memory compaction. Describe the steps involved in compaction and the main disadvantage that makes it undesirable in many systems. (CO1)
(PO1)
6. Consider a system with virtual memory using paging. The virtual address space is 2^{32} bytes, and the page size is 2^{12} (4 KB). The system has 64 MB of physical RAM.
- a) How many bits form the page number and how many the page offset? What is the maximum number of virtual pages per process? If each page table entry (PTE) occupies 4 bytes, what is the size of a singlelevel page table for a process that uses the entire virtual address space? Discuss why such a large page table is impractical. 2 +
2 + 6
(CO1)
(PO1)
- b) For an instruction of a process, the Memory Management Unit (MMU) translates the virtual address but finds that the corresponding page is not present in physical memory and one of the pages from the memory should be replaced to load the corresponding missing page. Describe step-by-step what happens from the moment the MMU detects the missing page until the instruction can be successfully restarted. 10
(CO1)
(PO1)
- c) Explain why a singlelevel page table is impractical for 64bit architectures. With a 64bit address and 4 KB pages, if each page table level consumes 10 bits, how many levels are required if we use a multilevel page table? 5 + 1
(CO
(PO